Bakgrund

Under hösten var vi arbetat med en Proof of Concept (PoC) för att testa möjligheterna att implementera en C-DAS lösning som bygger på SFERA, en europisk standard för C-DAS.

Ansatsen har varit att använda *FacilityData* från BIS som källa för anläggningsdata. *FacilityData* lämpar sig bra då den ligger närmare den modell vi vill använda internt, det mesta av data vi vill ha finns och det är en stabil lösning som MPK och SNTL använder sig av, vilket borde vara en fördel.

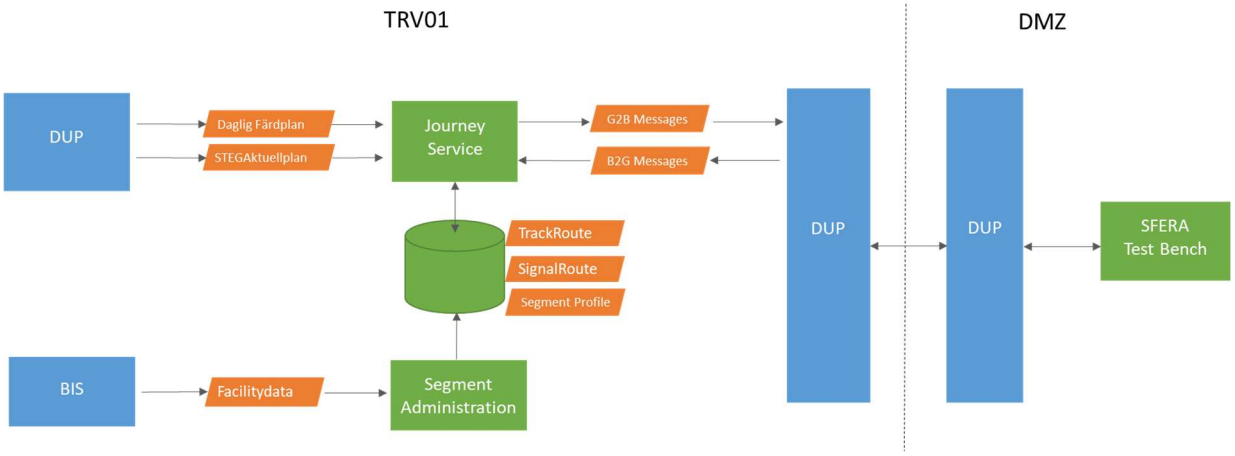
OperativFärdplan användes för att läsa in tåg/tidtabell och *STEGAktuellPlan* för uppdateringar av planen.

Proof of Concept

Under resans gång kom vi fram till att det skulle bli svårt att endast lagra *Segment Profile* i DUP och det skulle ge bättre kontroll att lagra dessa lokalt i vår databas.

Initialt så fanns det också idéer om att som ett delsteg låta järnvägsbolag integrera direkt mot DUP i DMZ. För att slippa integration mot DUP så har vi istället skapat en webb (SFERA Test Bench) som järnvägsbolagen kan använda för att testa konceptet och validera data som vi fyllt standarden med. Förhoppningen är att detta ska sänka tröskeln för dem att komma igång och komma med återkoppling.

Systemskiss



Avgränsningar

Vår lösning bygger på SFERA 2.0.
Vi kommunicerar endast med RU-DAS-TS, även om SFERA medger direkt kommunikation med tågen.
Vi stöder endast uppkoppling ”Connected” och skickar inga instruktioner utan bara data till RU (*BoardAdviceCalculation*).

Fortsättning

- Kommunicera data som skapas i PoC:en med SFERA Test Bench för att få återkoppling
- Vidareutveckla Journey Service med modulerna *TimingPointWorker* och *TågrapportWorker.* Implementera fler Use Case, även de som tar emot information från tågen.
- Visualisering och validering har inte realiserats i och kanske kan vara föremål för en egen PoC
- Få igång kommunikationen via CI
- Skapa en tjänst som automatiskt kan uppdatera *Segment Profiles*

Journey Service

Vi vidareutvecklar *Journey Service*. Den läser idag in *OperativFärdplan* och *STEGAktuellPlan* och skapar *Journey Profiles*. Dessa *Journey Profiles* är kopplade till *Segment Profiles* och *Train Characteristics*. Integration mot Topaz för att få en Tågrapporter behövs utvecklas samt en rutin som beräknar målpunkter.

Ett antal av de mest grundläggande användningsfallen (*SFERA Use Cases*) kring uppkoppling och hämtning data är implementerade, vi behöver fortsätta med att implementera resten av de vi har valt att stödja.

Moduler

OperativFärdplanWorker

Läser in Operativ Färdplan och skapar en första version av *Journey*

AktuellPlanWorker

Läser in *STEGAktuellPlan* och uppdaterar *Journey Profile* med tider och spår

TågrapportWorker

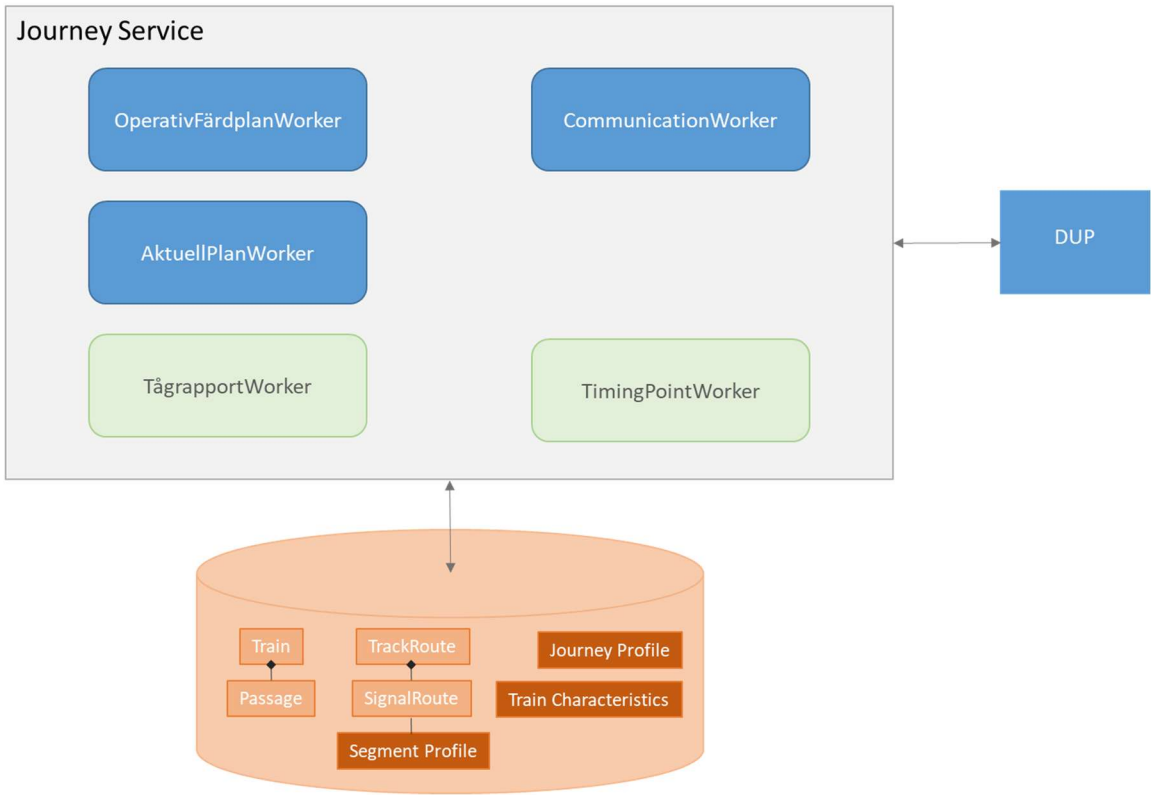
Läser in *Tågrapport* från Topaz och sparar tågsammansättningen för att sedan uppdatera *Journey Profile*

TimingPointWorker

Rutin som kontinuerligt beräknar målpunkter och vid förändringar uppdaterar *Journey Profile*

CommunicationWorker

Tar emot och skickar meddelanden enligt SFERA 2.0. Implementerar de flesta Use Case som finns beskrivna i SFERA.



Skapande och uppdatering av Journey Profile

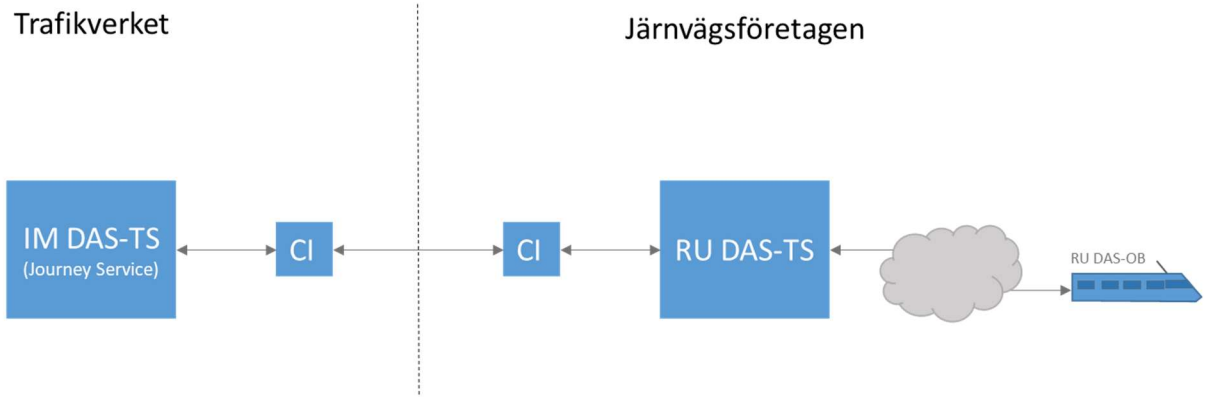
Vid inläsning av nya tåg och vid uppdateringar använder vi oss av stödtabellerna *Train* och *Passage*.

När nya tåg läses in av *OperativFärdplanWorker* skapas informationen. När uppdateringar kommer från *AktuellPlanWorker*, *TågrapportWorker*, *TimingPointWorker* eller *CommunicationWorker* uppdateras informationen i stödtabellerna.

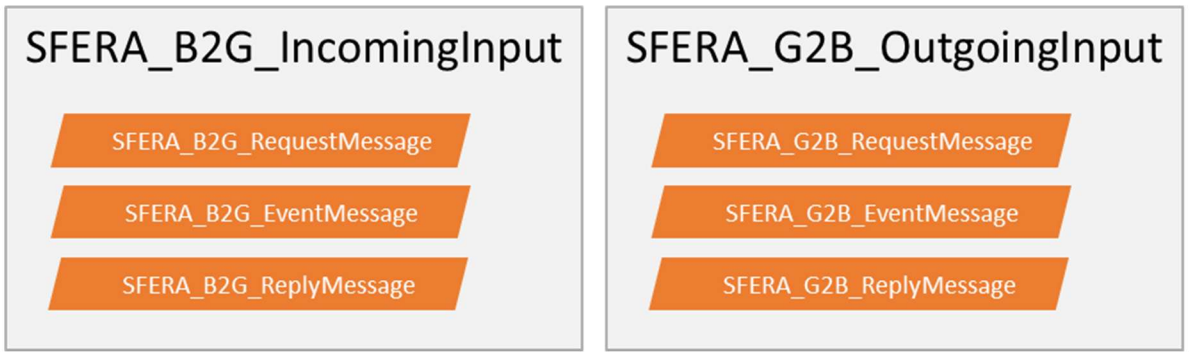
Stödtabellerna ligger sedan till grund för att skapa en *Journey Profile*.

Kommunikation

Trafikverkets har i C-DAS sammanhang en roll som Infrastructure Manager (IM). *Journey Service* är en service som driftsätts i Trafikverkets operativa miljö (TRV01) och är därmed Track Side (TS) i DAS termer. *Journey Service* är kärnan i vår lösning som kommunicerar via CI med Järnvägsföretagens DAS servrar (RU DAS-TS).



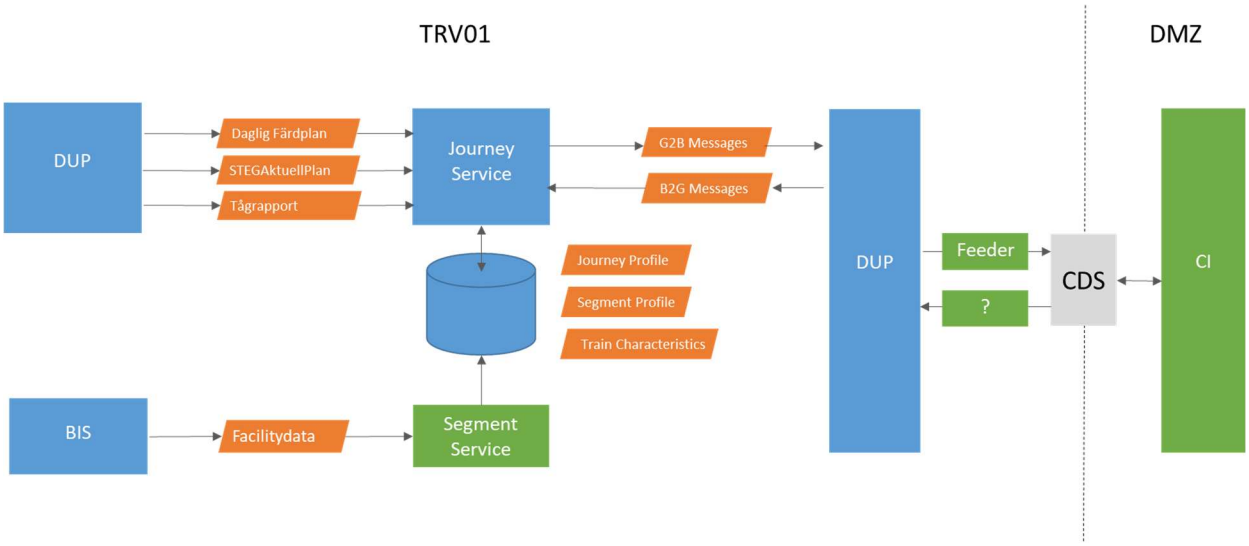
Extern kommunikationen (RU-DAS-TS) hanteras främst av *CommunicationWorker* och sker via sex meddelanden som är standardiserade i SFERA. Dessa meddelanden implementeras som två stycken kollektioner i DUP, en för inkommande meddelanden (B2G) och en för utgående (G2B).



Common Interface

I enlighet med SFERA standarden så ska kommunikationen ske via Common Interface (CI). Det är för tillfället oklart hur vi ska kommunicera med CI. Vår förhoppning är att andra ska bana väg för att skapa integration mellan DUP och CI som vi kan använda, t ex *CI_Feeder*.

Alternativet är att vi själva skriver till och läser från AMPQ.



Skiss på en möjlig koppling mellan DUP och CI

Segment Service

För att det ska finnas *Segment Profiles* att koppla *Journey Profiles* mot så har vi skapat en applikation som läser in anläggningsdata från BIS och producerar *Segment Profiles.* När jag kommer till ett driftläge måste *Segment Profiles* automatiskt uppdateras när anläggningsdata förändras. Detta åstadkommer vi genom att skapa *Segment Service*, en tjänst som dygnvis uppdaterar *Segment Profiles* från *FacilityData*-filer.

Indata

Facilitydata är det format som SNTL kommer att använda för anläggningsdata så vi antar att det kommer att vara tillgängligt i framtiden. Att vi använder samma källa som SNTL borde vara en fördel. Vi har använt *Facilitydata* i PoC:en

Datamängder

För att skapa en fungerande segmentprofil så har vi identifierat en miniminivå av information som den måste innehålla.

Det är attributen *SP_ID*, *SP_Zone*, *SP_VersionMajor*, *SP_VersionMinor*, *SP_Length*, *SP_Status* samt objekten *TimingPoint*, *StaticSpeedProfile*, *Line&TrackIdentifier*

- *SP_ID* Se Indelning och ID nedan
- *SP_Zone* Vi sätter IM_ID till 74 (som är Trafikverket)
- *SP_VersionMajor* & *SP_VersionMinor* Se Versionshantering nedan
- *TimingPoint* använda signalers positioner för att få mer exakta positioner
- *StaticSpeedProfile* Vi skapar en grundprofil som kan överridas med en specifik profil för just det tågslaget, här används *trainCategoryCode* med koppling till *Train Characteristics*. Profilerna kan beskrivas på olika sätt och behöver utredas mer
- *Virtual Balise* för att stödja positionering. Vi börjar med det som finns i *FacilityData,* behövs det fler koordinater får vi ta de från en annan källa
- *Line&TrackIdentifier* indentiteter på linje och stationsspår

Indelning och ID

Det går att dela in segmenten på ett antal olika sätt, alla med för och nackdelar. För att göra det ”enkelt” så vill jag skapa segmenten baserade på länkar. Länk är ett topologiskt element som beskriver en anslutning mellan två noder. Man kan då använda länkens id som är stabilt för att skapa ett unikt id (SP_ID) för segmentprofilen, det är bara att lägga på vilken riktningen på länken som profilen gäller.

Länkens id består av från nod, till nod och riktning. Exempelvis blir första länken (riktning 0) på Apt (294) från nod (1) till nod (2) följande:



En segmentprofil som följer länkens riktning skulle då få id:et 294:1:294:2:0**:0** och i motsatt riktning 294:1:294:2:0**:1**

Längden på segmentsprofilen (*SP_Length*) blir då samma som länkens längd.

Versionshantering

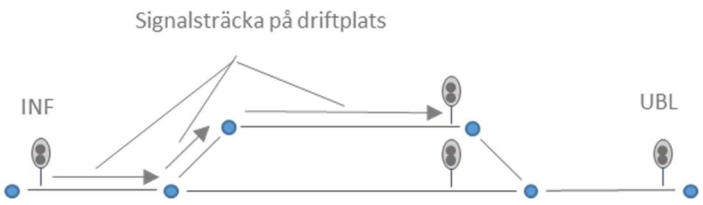
För att få versionerna att bara räknas upp när det sker förändringar så sparas en checksumma i databasen för major respektive minor.

SP_VersionMajor räknas upp om signifikant ändras. Till att börja med är det *SP_Length* och *TimingPoint.* Om *SP_VersionMajor* räknas upp sätts också *SP_VersionMinor* till 0.

SP_VersionMinor räknas upp om något över huvud taget i segmentsprofilen ändrats.

Signalsträckor

När vi skapar segmentprofiler baserade på länkar så har vi ett behov i Journey Service att hitta vilka segmentsprofiler som ingår i tågets väg. Eftersom STEG skickar ut STEGAktuellPlan med en lista av signaler som ingår i tågvägen så kan vi använda signalerna för att hitta vägen i nätet (länkarna) och därmed vilka segmentsprofiler som ingår. Vi behöver då generera Signalsträckor (*SignalRoute*) i Segment Service som *Journey Service* sedan kan använda för att koppla *Segment Profiles* till en *Journey Profile*. En signalsträcka är en sträcka mellan två signaler som kan visa stopbesked som innehåller en lista med ingående länkar. Eftersom *FacilityData* numera innehåller full täckning av tågvägar i Sverige så kan de användas för att översättas till Signalsträckor.



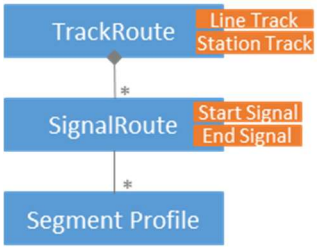


Spårsträckor

Vid initial inläsning av tåg och tidtabeller från *OperativFärdplan* har vi inga signaler som beskriver vägen. För att utifrån linjespår och stationsspår hitta segmentsprofiler och skapa *Journey Profiles* behöver vi också generera Spårsträckor som är kopplingen mellan spåren som anges i tidtabellen och Signalsträckorna.

Utdata

Förutom *Segment Profiles* så exporteras Signalsträckor (*SignalRoute*) och Spårsträckor (*TrackRoute*) till *Journey Service* för att möjliggöra översättning från tidtabeller i *OperativFärdplan* och *STEGAktuellPlan* till motsvarande *Segment Profiles.*



Entiteter som exporteras till databasen

Visning och validering

För att underlätta att förstå hur segmentprofilerna ser ut och bedöma dess riktighet så bör vi ta fram en visnings funktion om det inte redan finns i SFERA världen. Ska kunna visa enskilda segmentprofiler och en sekvens av segmentprofiler som ingår i en *Journey Profile*.

Någon slags funktion för att kontrollera täckning av segmentprofiler i nätet skulle vara bra. Detta blir relativt enkelt om vi har en direkt koppling mellan länkar och segmentprofiler. Eftersom identiteten för en segmentprofil byggs upp av ingående noders id så kan man lätt kontrollera om intill liggande segmentprofiler har en gemensam nod, om så är fallet ansluter de till varandra och det finns inget glapp.

Gränssnitt för att testa hantering av information som skickas från RU in till oss, kanske en utökning av SFERA Test Bench.

Det skulle också vara intressant att utveckla en enklare simulator som kan efterlikna ett tåg som är ute och kör.